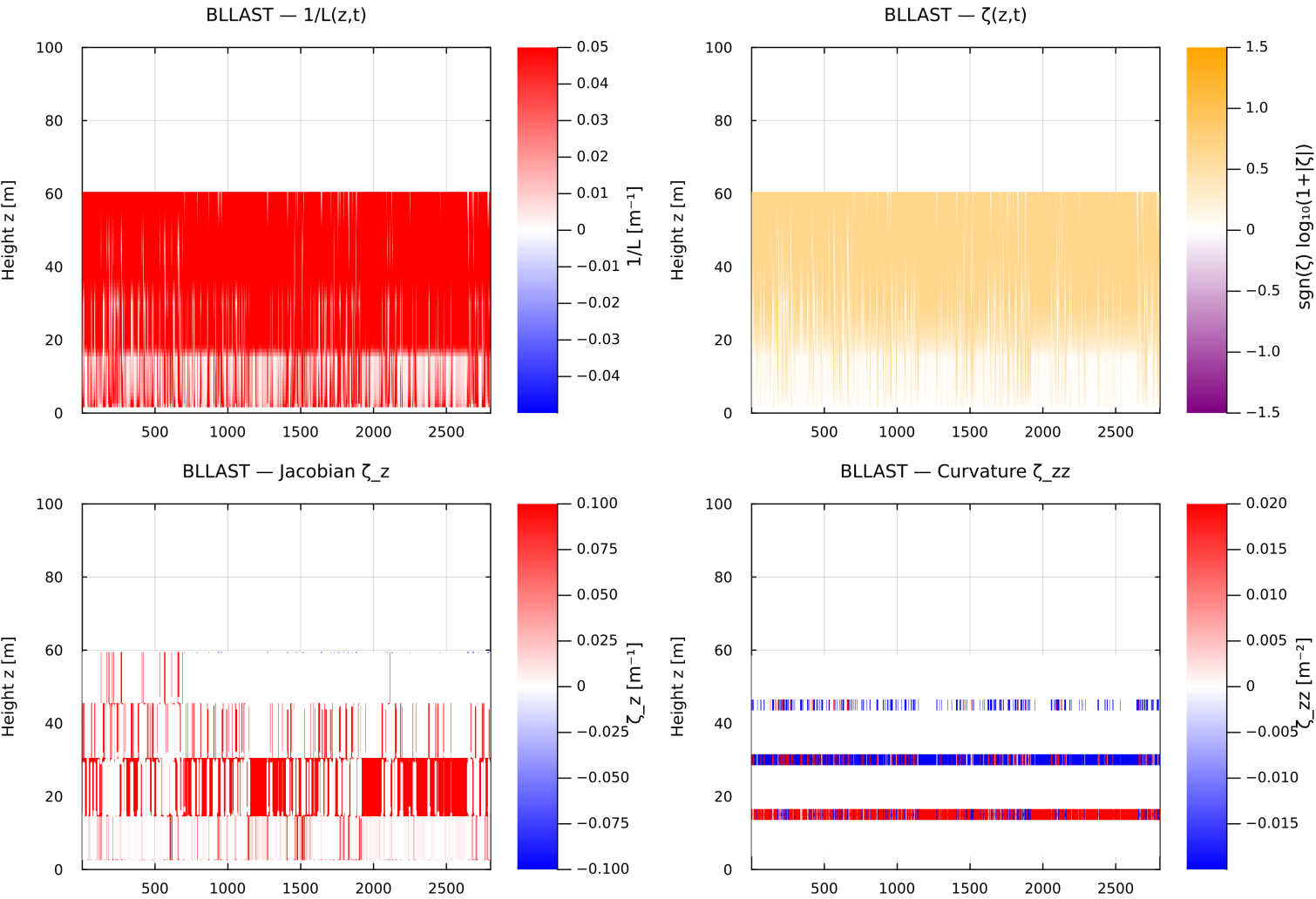
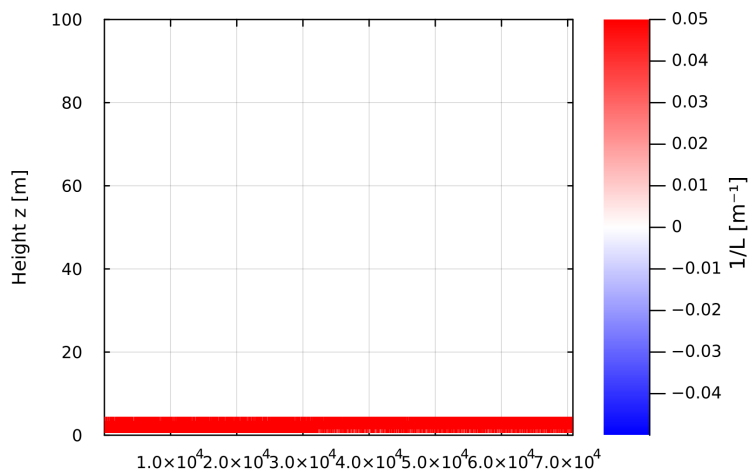
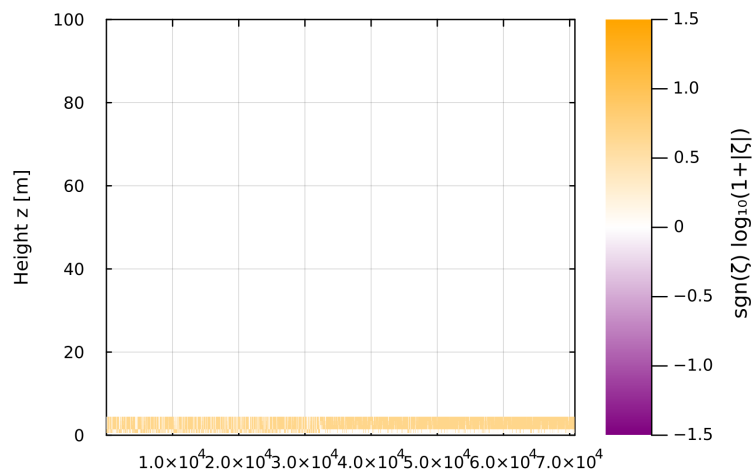
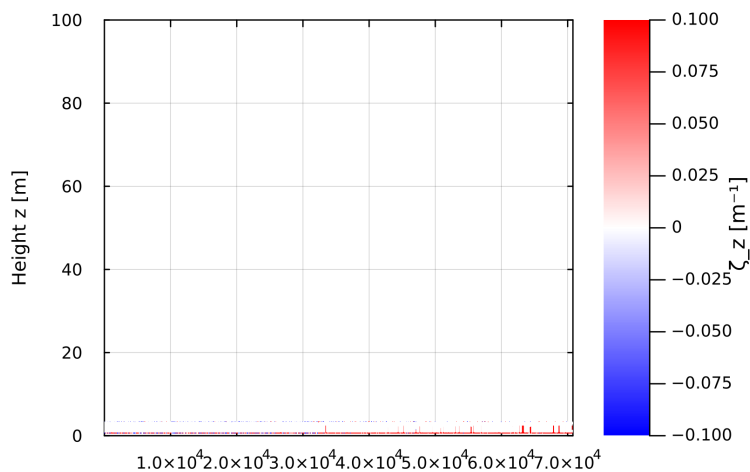
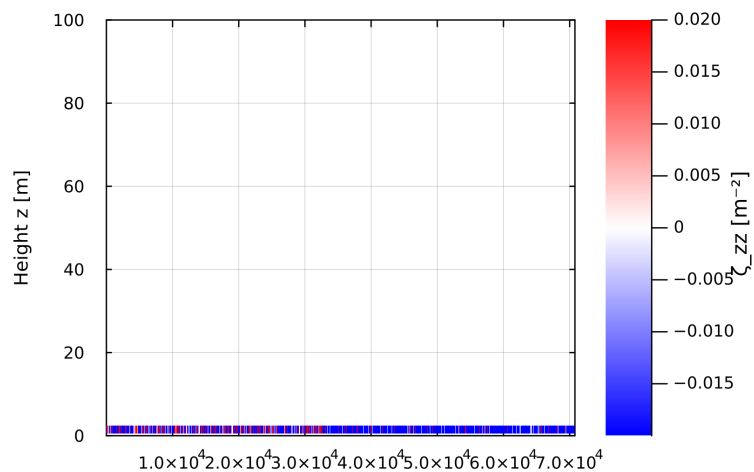
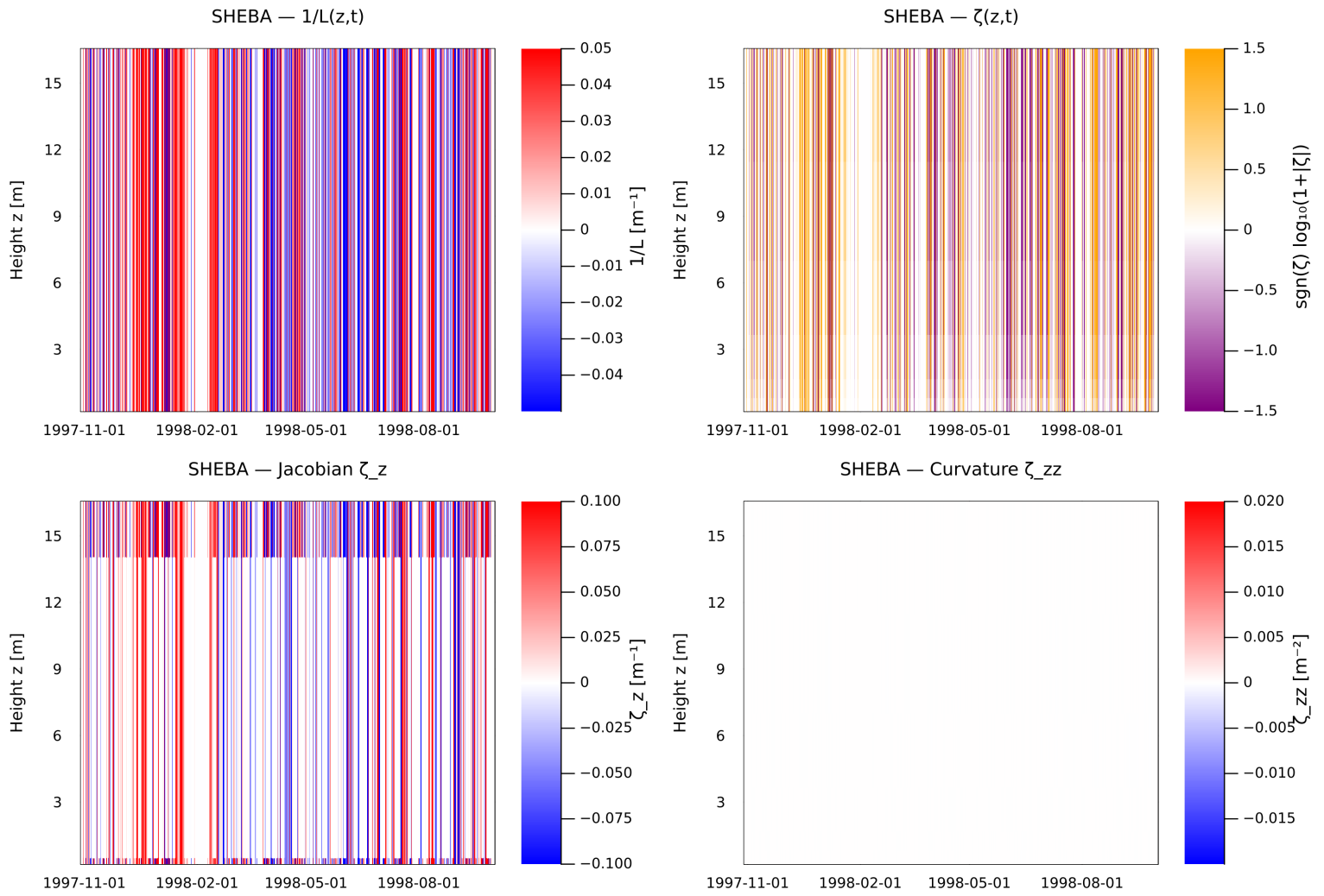


BLLAST

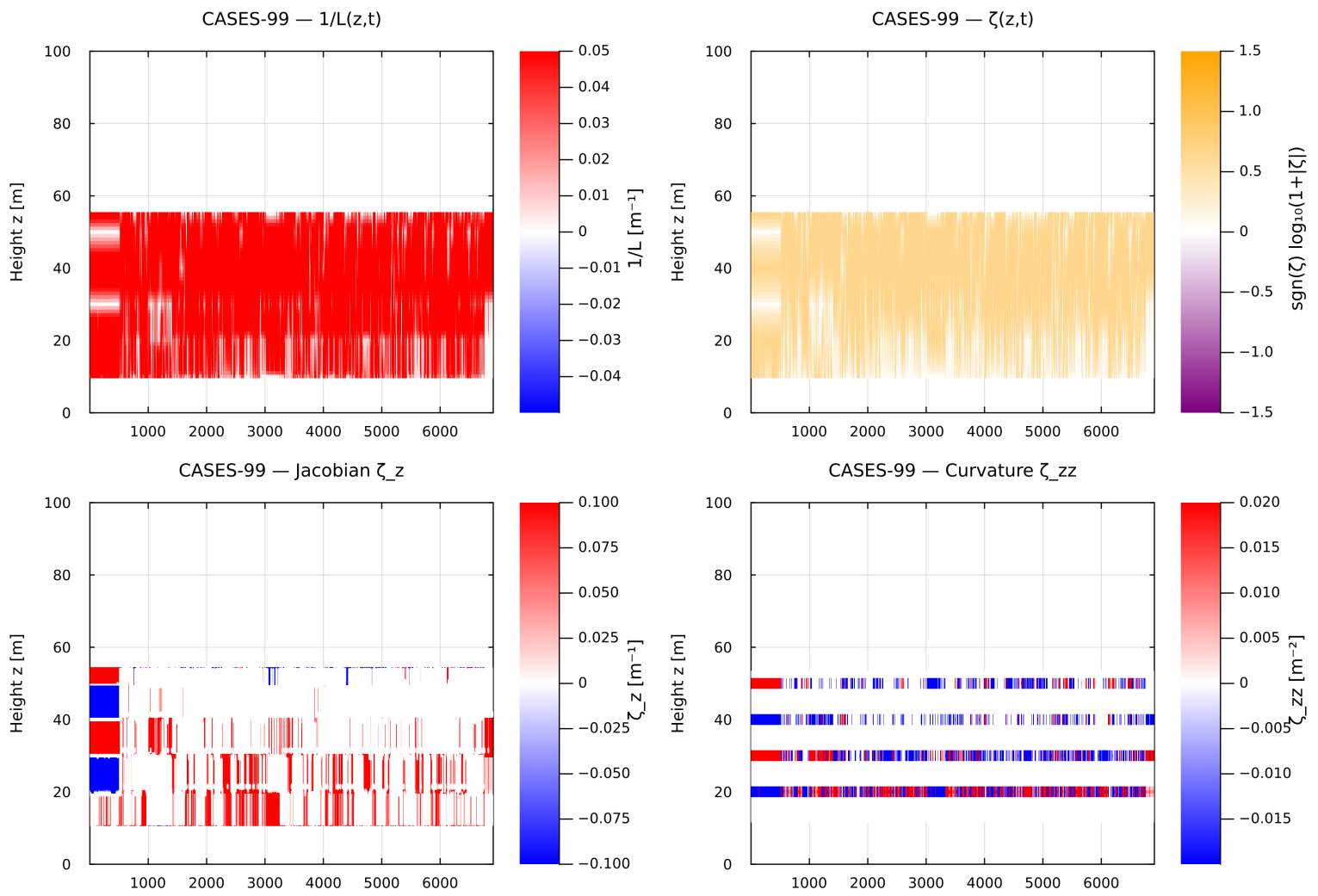


FLOSS II

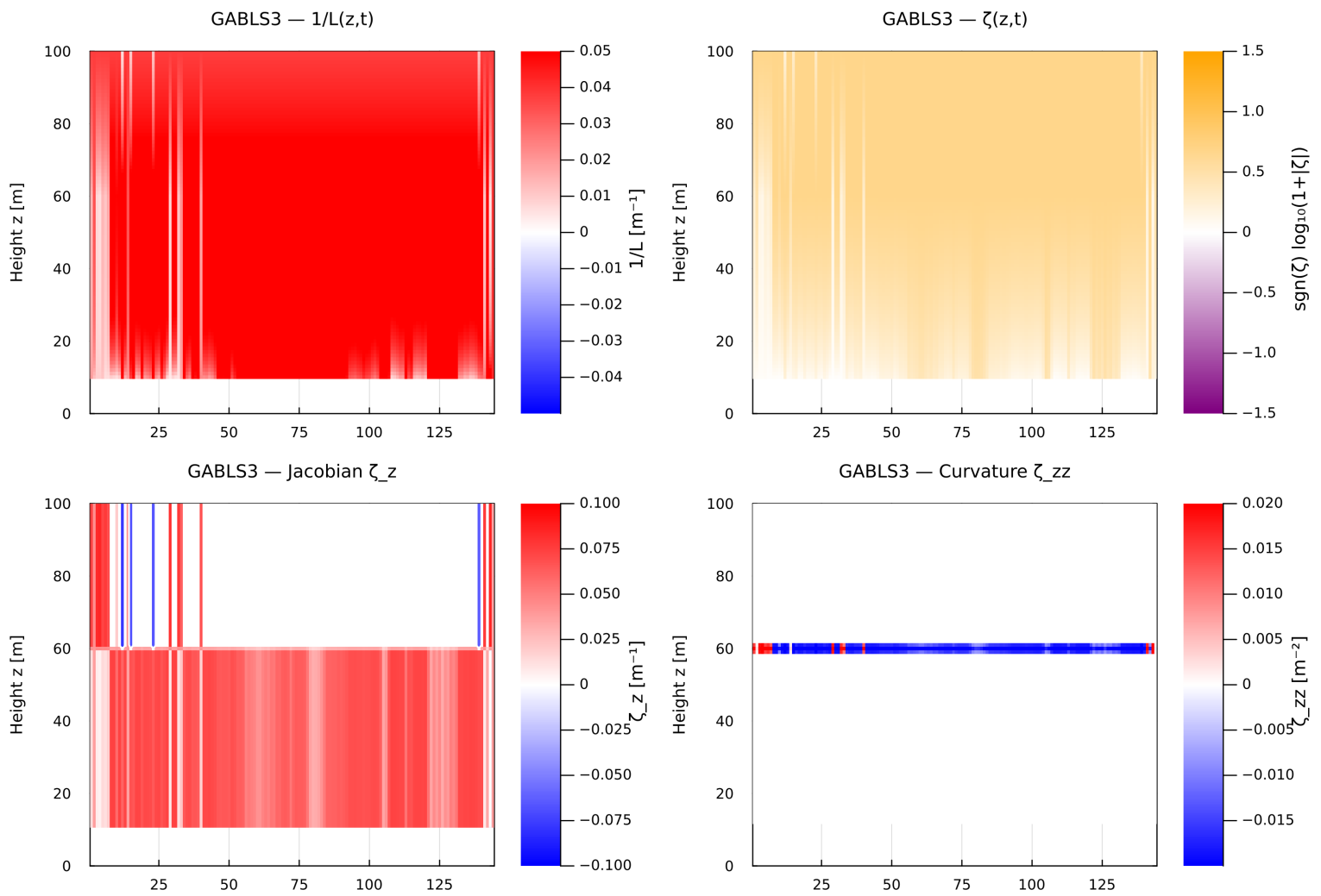
FLOSS_II — $1/L(z,t)$ FLOSS_II — $\zeta(z,t)$ FLOSS_II — Jacobian ζ_z FLOSS_II — Curvature ζ_{zz} 



CASES-99



GABLS3



Code

```
#!/usr/bin/env julia
# SBLToolkit.jl/scripts/plot_obukhov_heatmaps.jl

using Pkg
const PROJECT_ROOT = normpath(joinpath(@__DIR__, ".."))
Pkg.activate(PROJECT_ROOT)
push!(LOAD_PATH, joinpath(PROJECT_ROOT, "src"))

using SBLToolkit
using SBLToolkit.DatasetRegistry
using CSV
using DataFrames
using Dates
using NCDatasets
```

```

using Plots
using Plots.PlotMeasures
using Statistics

const SBL_OUTPUT_DIR = joinpath(PROJECT_ROOT, "reports", "generated", "sbltoolkit_heatr
mkpath(SBL_OUTPUT_DIR)

# Common vertical evaluation grid for standardized cross-campaign comparison
const COMMON_Z_GRID = Vector{Float64}(1.0:1.0:100.0)

slugify(s::String) = lowercase(replace(s, r"^[A-Za-z0-9]+" => "_"))
norm_str(s::Union{String,Symbol}) = lowercase(replace(string(s), r"^[a-z0-9]" => ""))

function safe_float(val)
    val === nothing && return NaN
    val === missing && return NaN
    val isa Number && return Float64(val)
    if val isa AbstractString
        parsed = tryparse(Float64, strip(val))
        return parsed !== nothing ? parsed : NaN
    end
    return NaN
end

function rig_to_zeta(rig::Float64)
    !isfinite(rig) && return NaN
    if rig >= 0.0
        rig_c = min(rig, 0.19) # Cap below critical Ri_g ~ 0.2 singularity
        return rig_c / (1.0 - 5.0 * rig_c)
    else
        return rig
    end
end

function parse_rig_height(col_name::Symbol)
    s = string(col_name)
    m = match(r"^ri_g_(\d+)_(\d+)$"i, s)
    if m !== nothing
        i_part = parse(Float64, m[1])
        d_part = parse(Float64, m[2])
        return i_part + d_part / (10^length(m[2]))
    end
end

```

```

end
m2 = match(r"^ri_g_(\d+)$"i, s)
if m2 !== nothing
    return parse(Float64, m2[1])
end
return nothing
end

# -----
# 1D Vertical Linear Interpolation to Standardized Grid
# -----
function interpolate_profile_1d(f_raw::Vector{Float64}, z_raw::Vector{Float64}, z_target::Vector{Float64})
    valid_idx = findall(isfinite, f_raw)
    length(valid_idx) < 2 && return fill(NaN, length(z_target))

    zr, fr = z_raw[valid_idx], f_raw[valid_idx]
    p = sortperm(zr)
    zr, fr = zr[p], fr[p]

    f_out = fill(NaN, length(z_target))
    for (k, zt) in enumerate(z_target)
        if zt < zr[1] || zt > zr[end]
            continue # Avoid unconstrained extrapolation
        end
        idx = searchsortedfirst(zr, zt)
        if idx == 1
            f_out[k] = fr[1]
        elseif idx <= length(zr)
            z0, z1 = zr[idx-1], zr[idx]
            f0, f1 = fr[idx-1], fr[idx]
            f_out[k] = f0 + (f1 - f0) * (zt - z0) / (z1 - z0)
        end
    end
    return f_out
end

# -----
# Target-Priority Two-Pass Matching
# -----
function match_fuzzy_col(cols::Vector{Symbol}, targets::Vector{String})
    norm_targets = norm_str.(targets)

```

```

norm_cols = norm_str.(cols)

for target in norm_targets
    for (i, nc) in enumerate(norm_cols)
        nc == target && return cols[i]
    end
end

for target in norm_targets
    length(target) < 3 && continue
    for (i, nc) in enumerate(norm_cols)
        occursin(target, nc) && return cols[i]
    end
end

return nothing
end

function match_nc_var(keys_list::Vector{String}, targets::Vector{String})
    norm_targets = norm_str.(targets)
    norm_keys = norm_str.(keys_list)

    for target in norm_targets
        for (i, nk) in enumerate(norm_keys)
            nk == target && return keys_list[i]
        end
    end

    for target in norm_targets
        length(target) < 3 && continue
        for (i, nk) in enumerate(norm_keys)
            occursin(target, nk) && return keys_list[i]
        end
    end

    return nothing
end

# -----
# Numerical Stencil Operators
# -----

```

```

function uniform_gradient_1d(f::Vector{Float64}, dz::Float64)
    n = length(f)
    df = fill(NaN, n)

    if n >= 2
        if isfinite(f[1]) && isfinite(f[2])
            df[1] = (f[2] - f[1]) / dz
        end
        if isfinite(f[end]) && isfinite(f[end-1])
            df[end] = (f[end] - f[end-1]) / dz
        end
    end

    for i in 2:(n-1)
        if isfinite(f[i-1]) && isfinite(f[i+1])
            df[i] = (f[i+1] - f[i-1]) / (2.0 * dz)
        end
    end

    return df
end

```

```

function uniform_hessian_1d(f::Vector{Float64}, dz::Float64)
    return uniform_gradient_1d(uniform_gradient_1d(f, dz), dz)
end

```

```

function assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
    nt = length(timestamps)
    z_target = COMMON_Z_GRID
    nz_target = length(z_target)
    dz = z_target[2] - z_target[1]

    # Interpolate input profiles onto standard z grid
    zeta_mat = fill(NaN, nz_target, nt)
    for j in 1:nt
        zeta_mat[:, j] = interpolate_profile_1d(raw_zeta_mat[:, j], raw_z_levels, z_target)
    end

    inv_L_mat = fill(NaN, nz_target, nt)
    for j in 1:nt, i in 1:nz_target
        if isfinite(zeta_mat[i, j])

```

```

        inv_L_mat[i, j] = zeta_mat[i, j] / z_target[i]
    end
end

zeta_z_mat = fill(NaN, nz_target, nt)
zeta_zz_mat = fill(NaN, nz_target, nt)
for j in 1:nt
    zeta_z_mat[:, j] = uniform_gradient_1d(zeta_mat[:, j], dz)
    zeta_zz_mat[:, j] = uniform_hessian_1d(zeta_mat[:, j], dz)
end

return (
    timestamps=timestamps,
    z_levels=z_target,
    inv_L=inv_L_mat,
    zeta=zeta_mat,
    zeta_z=zeta_z_mat,
    zeta_zz=zeta_zz_mat
)
end

# -----
# Parsers
# -----
function parse_obukhov_from_df(df::DataFrame, campaign_name::String)
    cols = propertynames(df)

    time_col = match_fuzzy_col(cols, ["sample_index", "sampleindex", "time_value", "tir
    time_col === nothing && error("No valid time column found.")

    raw_times = safe_float.(df[!, time_col])
    valid_time_mask = isfinite.(raw_times)
    timestamps = sort(unique(raw_times[valid_time_mask]))
    nt = length(timestamps)
    time_map = Dict{t => j for (j, t) in enumerate(timestamps)}
    row_time_indices = [get(time_map, t, 0) for t in raw_times]

    rig_cols = Symbol[]
    rig_heights = Float64[]
    for c in cols
        h = parse_rig_height(c)

```

```

        if h != nothing
            push!(rig_cols, c)
            push!(rig_heights, h)
        end
    end

end

if length(rig_cols) >= 3
    p = sortperm(rig_heights)
    raw_z_levels = rig_heights[p]
    sorted_cols = rig_cols[p]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    for (i, col) in enumerate(sorted_cols)
        col_vals = safe_float.(df[:, col])
        for r in eachindex(col_vals)
            t_idx = row_time_indices[r]
            if t_idx > 0
                raw_zeta_mat[i, t_idx] = rig_to_zeta(col_vals[r])
            end
        end
    end
end

@info "Parsed Multi-Level Ri_g profiles [$campaign_name]: $(count(isfinite, raw
return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
end

l_col = match_fuzzy_col(cols, ["L_obukhov", "lobukhov", "obukhovlength", "l"])
if l_col != nothing
    raw_z_levels = Float64[1.0, 2.0, 5.0, 10.0, 20.0, 40.0, 60.0, 80.0, 100.0]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    l_vals = safe_float.(df[:, l_col])
    for r in eachindex(l_vals)
        t_idx = row_time_indices[r]
        L_val = l_vals[r]
        if t_idx > 0 && isfinite(L_val) && abs(L_val) > 1e-4
            for i in 1:nz_raw
                raw_zeta_mat[i, t_idx] = raw_z_levels[i] / L_val
            end
        end
    end
end

```

```

        end
    end

    @info "Parsed 1D Surface L_obukhov [$campaign_name]"
    return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
end

error("No usable multi-level Ri_g profiles or L_obukhov columns in trajectory file.")
end

function search_nc_file(campaign_name::String)
    clean = replace(lowercase(campaign_name), r"^[a-z0-9]" => "")
    candidate_dirs = [
        joinpath(PROJECT_ROOT, "data", clean),
        joinpath(PROJECT_ROOT, "data", "raw", clean),
        joinpath(PROJECT_ROOT, "data")
    ]

    for d in candidate_dirs
        !isdir(d) && continue
        for (root, _, files) in walkdir(d)
            for f in files
                if endswith(f, ".nc") && (occursin("prof", lowercase(f)) || occursin("r
                    return joinpath(root, f)
                end
            end
        end
    end
    return nothing
end

function load_profile_from_nc(nc_path::String, campaign_name::String; g=9.81, theta_0=:
    !isfile(nc_path) && return nothing

    return NCDataset(nc_path, "r") do ds
        keys_list = collect(keys(ds))

        t_key = match_nc_var(keys_list, ["time", "datetime", "t", "sampleindex"])
        t_key === nothing && error("Missing time dimension in $nc_path.")
        timestamps = Float64.(vec(ds[t_key][:]))
    end
end

```

```

z_key = match_nc_var(keys_list, ["height", "heights", "z", "level", "levels", '
raw_z_levels = if z_key !== nothing
    Float64.(vec(ds[z_key][:]))
else
    @warn "No NetCDF height coordinate found for $campaign_name; using fallback
    Float64[0.5, 1.0, 2.0, 5.0, 10.0, 15.0, 20.0, 30.0, 45.0, 60.0, 80.0, 100.0]
end
u_star_key = match_nc_var(keys_list, ["ustar", "ustarm", "frictionvelocity"])
h_key = match_nc_var(keys_list, ["hs", "heatflux", "shf", "wt", "kinematicheatflux"])

(u_star_key === nothing || h_key === nothing) && error("Missing flux keys in NetCDF")

u_s = vec(Float64.(ds[u_star_key][:]))
raw_h = vec(Float64.(ds[h_key][:]))
nt, nz_raw = length(timestamps), length(raw_z_levels)
raw_zeta_mat = fill(NaN, nz_raw, nt)

```

```

for j in 1:nt
    wt = abs(raw_h[j]) > 5.0 ? raw_h[j] / rho_cp : raw_h[j]
    if isfinite(u_s[j]) && isfinite(wt) && abs(wt) > 1e-5
        L_val = -(u_s[j]^3 * theta_0) / (k_vk * g * wt)
        for i in 1:nz_raw
            raw_zeta_mat[i, j] = raw_z_levels[i] / L_val
        end
    end
end
end

```

```

@info "NetCDF Extracted Flux Profiles [$campaign_name]"
return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)

```

```
end
```

```
end
```

```
# -----
```

```
# Plotting with Locked Standardized Heights (0–100 m)
```

```
# -----
```

```
function plot_sbltoolkit_obukhov_panel(data::NamedTuple, campaign_name::String, out_path::String)
    t_axis = data.timestamps
    z_axis = data.z_levels

```

```

    opts = (ylabel="Height z [m]", ylims=(0.0, 100.0), framestyle=:box, guidefontsize=:small)

```

```

cg_bwr = cgrad([:blue, :white, :red])
cg_puor = cgrad([:purple, :white, :orange])

p1 = heatmap(t_axis, z_axis, clamp.(data.inv_L, -0.1, 0.1);
    clims=(-0.05, 0.05), color=cg_bwr, title="$(campaign_name) - 1/L(z,t)", colorbar=:right)

zeta_symlog = @. sign(data.zeta) * log10(1.0 + abs(data.zeta))
p2 = heatmap(t_axis, z_axis, zeta_symlog;
    clims=(-1.5, 1.5), color=cg_puor, title="$(campaign_name) -  $\zeta(z,t)$ ", colorbar=:right)

p3 = heatmap(t_axis, z_axis, clamp.(data.zeta_z, -0.2, 0.2);
    clims=(-0.1, 0.1), color=cg_bwr, title="$(campaign_name) - Jacobian  $\zeta_z$ ", colorbar=:right)

finite_rows = findall(i -> all(isfinite, view(data.zeta_z, i, :)), axes(data.zeta_z))
if !isempty(finite_rows)
    zeta_z_contour = data.zeta_z[finite_rows, :]
    if minimum(zeta_z_contour) <= 0.0 <= maximum(zeta_z_contour)
        contour!(p3, t_axis, z_axis[finite_rows], zeta_z_contour, levels=[0.0], color=:black)
    end
end

p4 = heatmap(t_axis, z_axis, clamp.(data.zeta_zz, -0.05, 0.05);
    clims=(-0.02, 0.02), color=cg_bwr, title="$(campaign_name) - Curvature  $\zeta_{zz}$ ", colorbar=:right)

fig = plot(p1, p2, p3, p4, layout=(2, 2), size=(1100, 750), dpi=300)
savefig(fig, out_path)
return fig
end

# -----
# Main Pipeline
# -----

function main()
    campaign_sources = discover_campaign_trajectories()
    @info "Discovered $(length(campaign_sources)) campaign trajectories."

    success_count = 0

    for (name, path) in campaign_sources
        @info "Processing $name..."
        obukhov_data = nothing
    end
end

```

```

if isfile(path)
  try
    df = CSV.read(path, DataFrame)
    obukhov_data = parse_obukhov_from_df(df, name)
  catch e
    @warn "Trajectory CSV parsing failed for $name: $e"
  end
end

if obukhov_data === nothing
  nc_file = search_nc_file(name)
  if nc_file !== nothing
    @info "Attempting NetCDF extraction for $name via $nc_file"
    try
      obukhov_data = load_profile_from_nc(nc_file, name)
    catch e
      @warn "NetCDF fallback failed for $name: $e"
    end
  end
end

if obukhov_data === nothing
  @warn "Skipping $name: No valid stability profile data found."
  continue
end

fig_out = joinpath(SBL_OUTPUT_DIR, "$(slugify(name))_obukhov_heatmaps.png")
plot_sbltoolkit_obukhov_panel(obukhov_data, name, fig_out)
@info "Successfully rendered standardized heatmap for $name -> $fig_out"
success_count += 1
end

@info "Completed heatmaps pipeline: $success_count/$(length(campaign_sources)) rendered"

if abspath(PROGRAM_FILE) == @__FILE__
  main()
end

```